



Timing-Safe and Latency-Insensitive Hardware Design Methodologies

Ruijie Gao



Hardware Design 101

- HDL (Verilog/VHDL)
 - The "source code" of a chip
 - A synthesizer compiles it to a netlist of gates, roughly the way a compiler lowers to assembly
- RTL (Register-Transfer Level)
 - The abstraction everyone actually writes at: values flow through wires (combinational, no memory) and land in registers (state, updated on the clock edge)
- The clock is a global discrete time step
 - Every register in the design updates in lockstep on each tick. "Cycle N" means "the N-th tick"
- Synthesis is permissive:
 - Ill-timed code compiles fine, produces a chip,

Hardware Design Is Where Software Was Before Type Systems

- Imagine writing C where every function's calling convention lives in a Word doc
 - Not its signature
- Calling at the "wrong moment" returns garbage silently
 - No crash, no warning
- The “compiler” accepts it.
 - You find the bug after taping out a \$10M chip
- That's Verilog. That's how the industry ships silicon.

When Is This Wire Meaningful?

- A hardware module = input wires + output wires + a hidden assumption
 - About when each is valid
- Example
 - "Give me two numbers on cycle 0, I'll return the product on cycle 3, don't touch my inputs in between"
- None of that is in the code
 - It lives in a PDF, or in an engineer who left three years ago

```
module foo (  
    input          clk,  
    input  [31:0]  a,  
    input  [31:0]  b,  
    output [31:0]  out  
);  
...  
endmodule
```

Every Pain Point Has a PL Analogue We Already Solved

Hardware question	PL concept
"When is this value valid?"	Lifetimes (Rust)
"Can I call this now?"	Typestate / session types
"Does composition preserve the contract?"	Linearity
"Behavior depends on invisible scheduling"	Effects / data races

- HDLs were designed in the 1980s by EE people to describe circuits, not by PL people to describe programs
- Moore's Law papered over it: one engineer could hold the whole chip in their head
- That era is over. Modern SoCs: hundreds of IPs, dozens of vendors, nobody sees the whole picture

Timing-Safe & Latency-Insensitive, in PL Terms

- Latency-insensitive design = `async/await` for circuits
 - A valid/ready handshake transfers data whenever both sides are ready
- Timing-safe HDLs = type systems for that model
 - Pull the timing contract out of the PDF and into the function signature
 - Compiler rejects ill-timed programs before synthesis

Foundation of Latency Insensitivity (Carloni et al., 2001)

- **Problem**

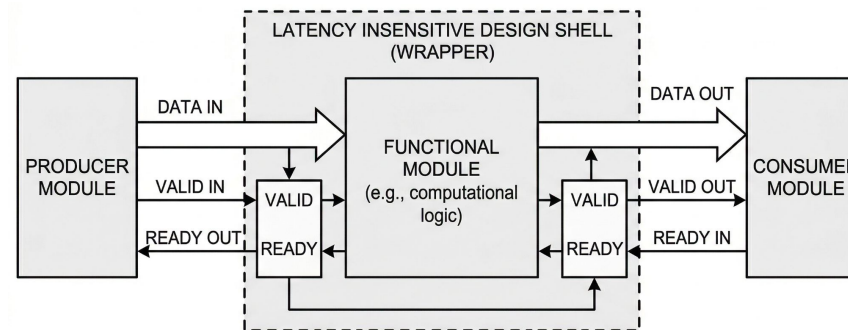
- Unpredictable wire delays breaking traditional synchronous design assumptions

- **Solution**

- Wrap synchronous modules and connect them using pipelined "relay stations".
- Guarantee functional correctness regardless of variable channel delays.

- **Evaluation & Limitations**

- Proven robust on a cycle-accurate model of the PDLX microprocessor
- Adding extra communication latency inevitably degrades the overall instruction throughput.



Unifying the Design Methodology (Lockhart et al., 2014)

- **Problem**

- Different languages, design patterns, and tools
- Different abstractions (functional-level (FL), cycle-level (CL), and register-transfer-level (RTL))

- **Solution**

- PyMTL is a unified Python-based framework for FL, CL, and RTL hardware modeling.
- It enforces latency-insensitive interfaces at every module boundary
 - models at different abstraction levels can be seamlessly composed and co-simulated.

- **Evaluation & Limitation**

- Evaluated on a dot-product accelerator tile and a 64-node mesh network
- ~4× slower than hand-written C++ for CL models and ~6× slower than Verilator-compiled Verilog for RTL models

Verifying LI Designs (Pan & Batten, 2023)

- **Problem**

- Bugs in latency-insensitive RTL modules are difficult to detect via standard simulation
- They only trigger under specific, complex combinations of stall cycles

- **Solution**

- Bounded Latency Equivalence Checking (BLEC)
- Verify if a module's egress outputs remain invariant across different ingress and egress stall conditions

- **Evaluation & Limitations**

- Identified subtle bugs in a processing element, a GCD unit, and a RISC-V processor
- Achieving a full bounded proof of correctness required manual design simplifications



Instead of finding bugs during verification,
can we eliminate them at compile time?



Filament: Modular Hardware Design with Timeline Types

- **Problem**

- HDL interfaces expose only port names and bitwidths, timing is hidden in docs or source
- Latency-insensitive wrappers work but add prohibitive overhead

- **Solution: Timeline Types**

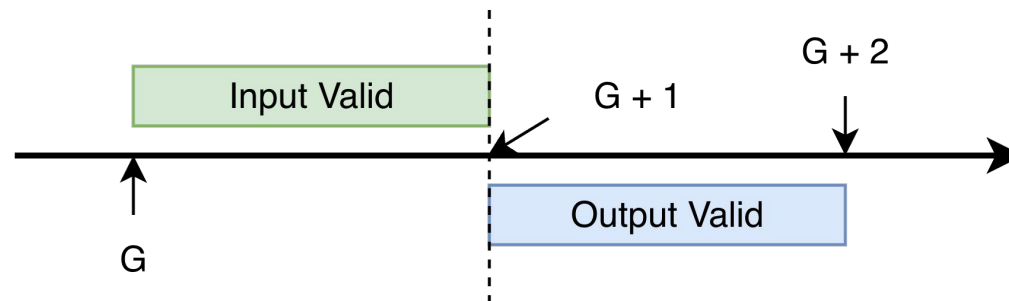
- Signature tells you everything about when signals move

```
extern comp Reg<G: 1>(@[G, G+1] in: 32) -> (@[G+1, G+2] out: 32);
```

Accepts new inputs
every cycle (initiation
interval = 1)

Input must be valid
for 1 cycle starting
at event G

Output value
appears one cycle
later



Catching an ALU Bug Using Timeline Types

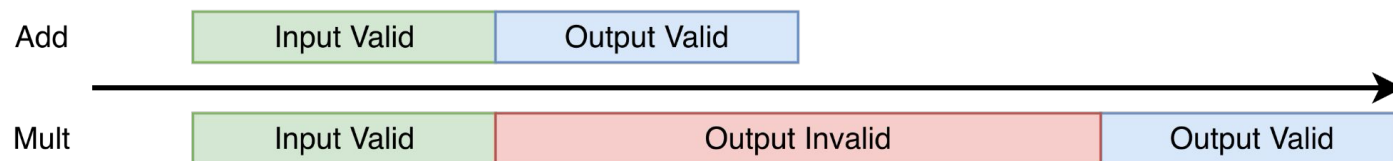
```
extern comp Add<T: 1>(@[T, T+1] l, r: 32) -> (@[T, T+1] out: 32);  
extern comp Mult<T: 3>(@[T, T+1] l, r: 32) -> (@[T+2, T+3] out: 32);
```

- Adder: combinational, result same cycle as inputs
- Multiplier: 2-cycle latency, new inputs every 3 cycles

```
a0 := A<G>(l, r);           // adder output at [G, G+1)  
m0 := M<G>(l, r);           // multiplier output at [G+2, G+3)  
mux := Mux<G>(op, m0.out, a0.out); // type error
```

Compiler rejects it

"m0.out available for [G+2, G+3) but required during [G, G+1)"



Catching an ALU Bug Using Timeline Types

```
extern comp Add<T: 1>(@[T, T+1] l, r: 32) -> (@[T, T+1] out: 32);
extern comp Mult<T: 3>(@[T, T+1] l, r: 32) -> (@[T+2, T+3] out: 32);
```

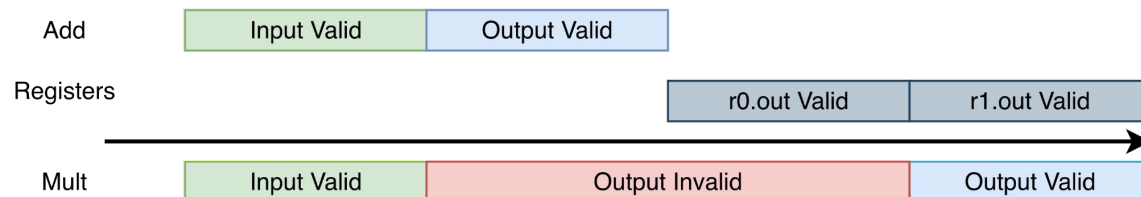
- Adder: combinational, result same cycle as inputs
- Multiplier: 2-cycle latency, new inputs every 3 cycles

```
a0 := A<G>(l, r);           // adder output at [G, G+1)
m0 := M<G>(l, r);           // multiplier output at [G+2, G+3)

r0 := R0<G>(a0.out);        // [G+1, G+2)
r1 := R1<G+1>(r0.out);      // [G+2, G+3)
mux := Mux<G+2>(op, r1.out, m0.out); // aligned
```

Type checks

Now both mux inputs are valid at [G+2, G+3)



Evaluation

- Wrote timeline types for 14 designs from Aetherling (a streaming DSL) and PipelineC (a C-to-hardware compiler)]

Design	Logic cells	Clock rate (MHz)
Aetherling	104	769
Filament	128	833
Filament+Reticle	14	645

- Filament compiles to hardware competitive with a specialized streaming DSL
- When composed with lower-level primitives (Reticle), **~7× fewer** logic cells

Limitation

- Only handles pipelines whose timing is fixed at compile time
 - What if data-dependent?
- All interval endpoints must be compile-time constants
 - No dependent types over runtime values
- Type system proves timing safety (no stale reads, no resource conflicts), not functional correctness of the computation itself

Parameterized Timing Contracts (Nigam et al., 2024 - Lilac)

- **Problem**

- Filament's types are fixed at design time, but generators (FloPoCo, Vivado IP, Aetherling) choose latencies based on their inputs (bitwidth, FPGA target, frequency goal)
- HDL parameters only flow parent to child; generated modules need to pass timing info upward

- **Solution**

```
gen "flopoco" comp FPAdd[#W]<G:1>(l, r: [G, G+1] #W)  
-> (o: [G+#L, G+#L+1] #W) with { some #L; }
```

#L is returned by the generator
at elaboration time

Intervals become symbolic ($[G+Add::\#L, \dots]$); an SMT solver proves
every parameterization is hazard-free

Parameterized Timing Contracts (Nigam et al., 2024 - Lilac)

- **Problem**

- Filament's types are fixed at design time, but generators (FloPoCo, Vivado IP, Aetherling) choose latencies based on their inputs (bitwidth, FPGA target, frequency goal)
- HDL parameters only flow parent to child; generated modules need to pass timing info upward

- **Solution**

```
gen "flopoco" comp FPAdd[#W]<G:1>(l, r: [G, G+1] #W)  
  -> (o: [G+#L, G+#L+1] #W) with { some #L; }
```

- **Evaluation & Limitation**

- 26% fewer LUTs, 33% fewer registers, 6.8% higher frequency vs. ready/valid Verilog
- Still static pipelines only

Dynamic Timing Contracts (Yu et al., 2026 - Anvil)

- **Problem**

- Real hardware has data-dependent timing: caches (hit vs. miss), page table walkers, variable-latency dividers.
- Wrapping them in conservative worst-case contracts throws away the whole point of caching

- **Solution: Events as Abstract Time Points**

from the request event until the next response event

```
chan cache_ch {  
  right req  : (addr @ req->res),      // valid until response arrives  
  left  res  : (data @ res->res+1)     // valid until next response  
}
```

- **Evaluation & Limitations**

- Expressed 10 real designs impossible in Filament: CVA6 page table walker, OpenTitan AES, AXI-Lite routers, caches
- vs. hand-written SystemVerilog: 4.5% area, 3.75% power, zero cycle-latency overhead
- Events must be statically relatable in the event graph

Open Research Questions

- LI + Timeline Type Check
 - Generate LI modules as a starting point
 - Prune the LI protocol overhead (FSM & Logic)

Open Research Questions

- LI + Timeline Type Check
- The Static vs. Dynamic Verification Dilemma
 - Existing timing-safe languages only solve design-time timing abstractions
 - Real-world hardware heavily relies on dynamic execution conditions
 - To bridge this gap by introducing a new verification and synthesis methodology
 - LTL Synthesis
 - Primitive Semantics Based DSL



Thank you! Any questions?

